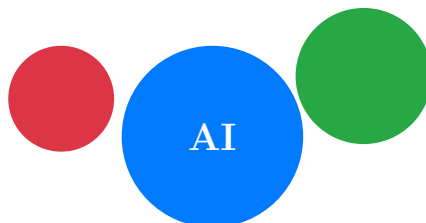


自然语言处理课程笔记

第 7 章：Transformer 与大语言模型

——多头注意力机制与架构深度解析



整理人：林俊杰

时间：2025 年 12 月 16 日

目录

1 多头注意力机制 (Multi-Head Attention)	2
1.1 注意力机制的动机 (Motivation)	2
1.2 输入表示: 词向量 + 位置编码	2
1.3 核心组件: Q、K、V 的物理含义	3
1.4 缩放点积注意力 (Scaled Dot-Product Attention)	3
1.5 多头注意力 (Multi-Head Attention)	4
2 Transformer 架构 (The Transformer Architecture)	6
2.1 整体架构可视化	6
2.2 解码器 (Decoder) 的独特之处	6

1 | 多头注意力机制 (Multi-Head Attention)

1.1 注意力机制的动机 (Motivation)

在传统的神经语言模型（如 RNN/LSTM）中，处理长序列时存在显著瓶颈：

- **长距离依赖丢失**：预测当前词 x_t 时，前面的词 $(x_1, \dots, x_{t-1})$ 对当前词的贡献被“平均化”或遗忘。
- **无法并行**：RNN 必须按时间步顺序计算，无法利用 GPU 并行加速。

注意 / 核心思想

直观理解：阅读句子 “The knight attacked the windmill on his donkey” 时，对于 “attacked” 这个词，我们的大脑会重点关注 “knight”（施动者）和 “windmill”（受动者），而忽略 “the”、“on” 等虚词。注意力机制就是模拟这种**动态分配权重**的过程。

1.2 输入表示：词向量 + 位置编码

Transformer 摒弃了循环结构，因此必须显式地注入位置信息。

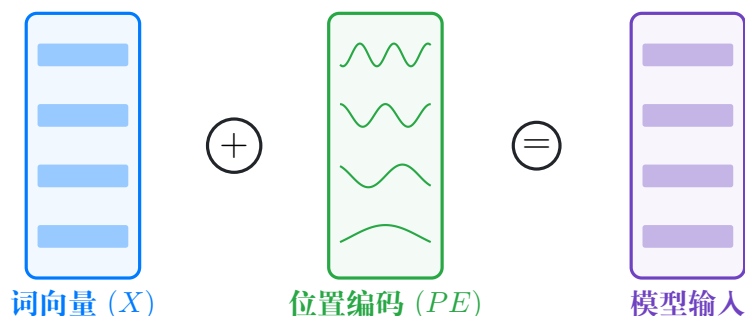


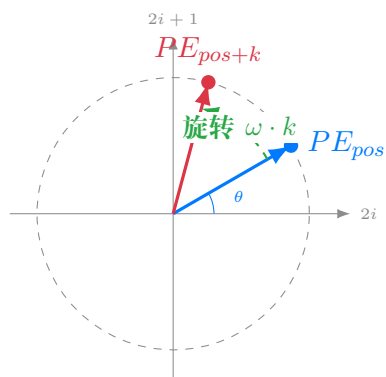
图 1: 位置注入机制：词向量与位置编码按元素直接相加 (Element-wise Addition)

核心公式：位置编码 (Positional Encoding, PE)

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (1)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (2)$$

- pos : 单词在句子中的绝对位置。
- i : 维度的索引， d 是模型的总维度。
- **特性**：这种设计允许模型学习相对位置，因为 PE_{pos+k} 可以表示为 PE_{pos} 的线性函数。



线性性质可视化:

位置 $pos + k$ 的向量可视为
位置 pos 的向量旋转了角度 ωk

$$PE_{pos+k} = \underbrace{\begin{pmatrix} \cos(\omega k) & -\sin(\omega k) \\ \sin(\omega k) & \cos(\omega k) \end{pmatrix}}_{R_k} \cdot PE_{pos}$$

图 2: 相对位置编码特性: PE_{pos+k} 可由 PE_{pos} 经旋转得到, 允许模型学习相对位置

1.3 核心组件: Q、K、V 的物理含义

输入向量经过三个线性变换矩阵 (W^Q, W^K, W^V) 映射为三个向量:

搜索引擎类比

- **Query (q): 查询向量。**代表“我在找什么?” (类似于在搜索引擎输入的关键词)。
- **Key (k): 键向量。**代表“我有什么特征?” (类似于网页的标题或标签, 用于被匹配)。
- **Value (v): 值向量。**代表“我的实际内容是什么?” (类似于网页的具体内容)。

Attention Score: 关键词与标题的匹配度, 决定了网页内容的展示权重。

1.4 缩放点积注意力 (Scaled Dot-Product Attention)

核心公式: 计算公式

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (3)$$

1. **线性投影:** 输入 x 乘以 W^Q, W^K, W^V 得到 q, k, v 。
2. **计算相似度:** 计算 q 与所有 k 的点积 ($q \cdot k^T$)。点积越大, 相关性越高。
3. **缩放 (Scaling):** 除以 $\sqrt{d_k}$ 。
 - 原因: 防止点积结果过大, 导致 Softmax 进入饱和区 (梯度接近 0, 难以训练)。
4. **归一化 (Softmax):** 将分数转换为概率分布 (总和为 1), 即注意力权重。
5. **加权求和:** 用权重对 v 进行加权求和, 得到最终的上下文向量 (Context Vector)。

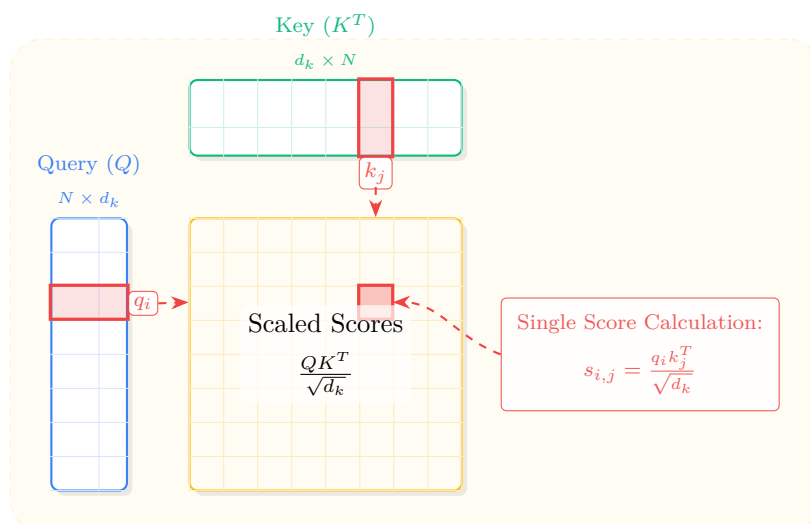


图 3: 缩放点积注意力机制的计算流程示意图

1.5 多头注意力 (Multi-Head Attention)

多头注意力的优势

如果只使用一个注意力头，模型可能只能关注到一种类型的相关性（例如，语法上的主谓关系）。多头注意力机制通过将 Query, Key, Value 线性投影到多个不同的低维子空间中，并在这些子空间中并行地计算注意力，使得模型能够：

- 捕捉不同模式的依赖关系（例如，一个头关注句法，另一个头关注语义）。
- 增强模型的表达能力和鲁棒性。

核心公式：多头注意力公式

假设头数为 h ，输入维度为 d_{model} ，单头维度 $d_k = d_{model}/h$ 。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (4)$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (5)$$

其中， $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{model} \times d_k}$ 是第 i 个头的投影矩阵， W^O 是最终的输出投影矩阵。

工程实现：整体投影与逻辑拆分

在工程实现中，为了最大化 GPU 并行效率，多头注意力并不物理分割为多个独立的矩阵，而是通过以下步骤实现：

- **整体投影 (Fused Projection)**：使用单个大权重矩阵一次性计算所有头的的数据。
- **逻辑拆分 (Logical Split)**：通过维度变换 (Reshape) 将结果在逻辑上划分为多个头。
- **并行计算**：利用广播机制 (Broadcasting) 同时对所有头进行注意力计算。

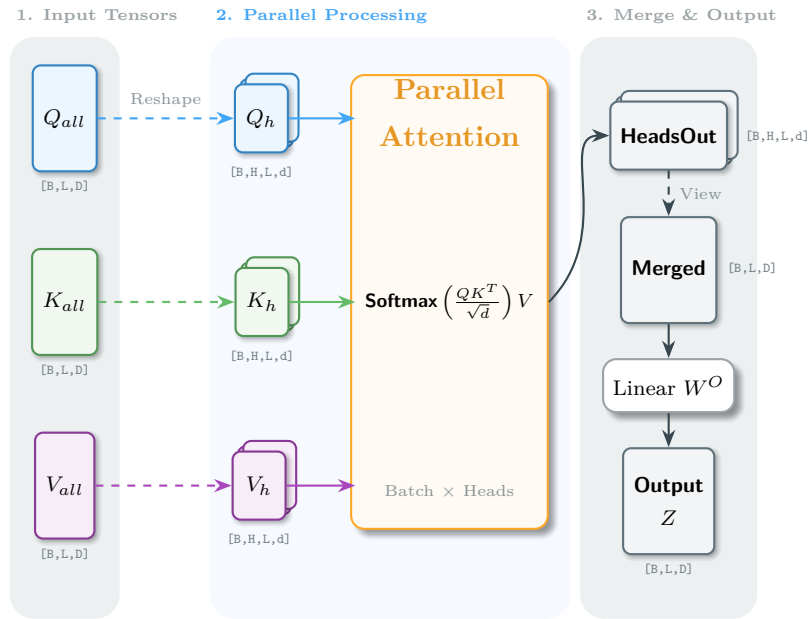


图 4: 多头注意力的计算流程示意图

核心公式：计算流程 (Tensor Operations)

假设输入 $X \in \mathbb{R}^{B \times L \times d_{model}}$ (B : Batch, L : Seq Length)。

1. 线性投影 (Linear Projection):

使用全尺寸矩阵 $W^Q, W^K, W^V \in \mathbb{R}^{d_{model} \times d_{model}}$ 进行一次性变换:

$$Q_{flat} = XW^Q, \quad K_{flat} = XW^K, \quad V_{flat} = XW^V$$

2. 维度重塑与转置 (Reshape & Transpose):

将 d_{model} 拆分为 $h \times d_k$, 并交换维度以便并行计算:

$$\text{View: } [B, L, d_{model}] \xrightarrow{\text{Reshape}} [B, L, h, d_k] \xrightarrow{\text{Transpose}} [B, h, L, d_k]$$

此时, 得到多头张量 $Q_{multi}, K_{multi}, V_{multi}$ 。

3. 并行注意力 (Parallel Attention):

$$\text{Out}_{multi} = \text{Softmax} \left(\frac{Q_{multi} K_{multi}^T}{\sqrt{d_k}} \right) V_{multi}$$

4. 拼接与输出 (Merge & Output):

逆向操作还原维度, 并经过最终线性层 W^O :

$$[B, h, L, d_k] \xrightarrow{\text{Transpose}} [B, L, h, d_k] \xrightarrow{\text{Reshape}} [B, L, d_{model}]$$

$$\text{FinalOutput} = \text{ConcatResult} \times W^O$$

2 | Transformer 架构 (The Transformer Architecture)

Transformer 是一个完全基于注意力机制的 **Encoder-Decoder**（编码器-解码器）架构。

2.1 整体架构可视化

Transformer 的主要特点包括：

1. **纯注意力机制**：完全依赖注意力机制来建模输入和输出之间的全局依赖关系，实现了高度的并行化计算。
2. **位置编码 (Positional Encoding)**：由于没有循环结构，模型无法感知输入序列的顺序。因此，需要显式地将位置信息（通常使用正弦和余弦函数生成）注入到输入嵌入中。
3. **编码器-解码器结构**：
 - **编码器**：由 N 个相同的层堆叠而成。每层包含两个子层：多头自注意力层和前馈神经网络层。
 - **解码器**：同样由 N 个相同的层堆叠而成。除了编码器中的两个子层外，还增加了一个**编码器-解码器注意力层**，用于关注编码器的输出。
4. **残差连接与层归一化**：每个子层都采用了残差连接和层归一化，这有助于训练深层网络并加速收敛。

2.2 解码器 (Decoder) 的独特之处

解码器相较于编码器，有两个主要的区别：

1. **掩码多头自注意力层 (Masked Multi-Head Self-Attention)**：

注意 / 核心思想

Mask 的作用 在训练时，模型在预测当前位置 t 的输出时，不能“偷看”到未来 (t 之后) 的位置信息。因此，需要在自注意力计算中应用一个掩码 (Mask)，将未来位置的注意力分数设置为 $-\infty$ (经过 Softmax 后变为 0)，从而确保预测只依赖于已知的历史信息。

2. **编码器-解码器注意力层 (Encoder-Decoder Attention)**：

- **Query (Q)**：来自解码器前一层的输出，代表当前生成步骤的查询意图。
- **Key (K) & Value (V)**：来自**编码器**的最终输出，代表源序列的完整语义信息。
- **作用**：这种机制让解码器在生成的每一步都能“回头看”源句子，动态地从编码器的输出中提取与当前生成任务最相关的信息（类似于机器翻译中的对齐机制）。

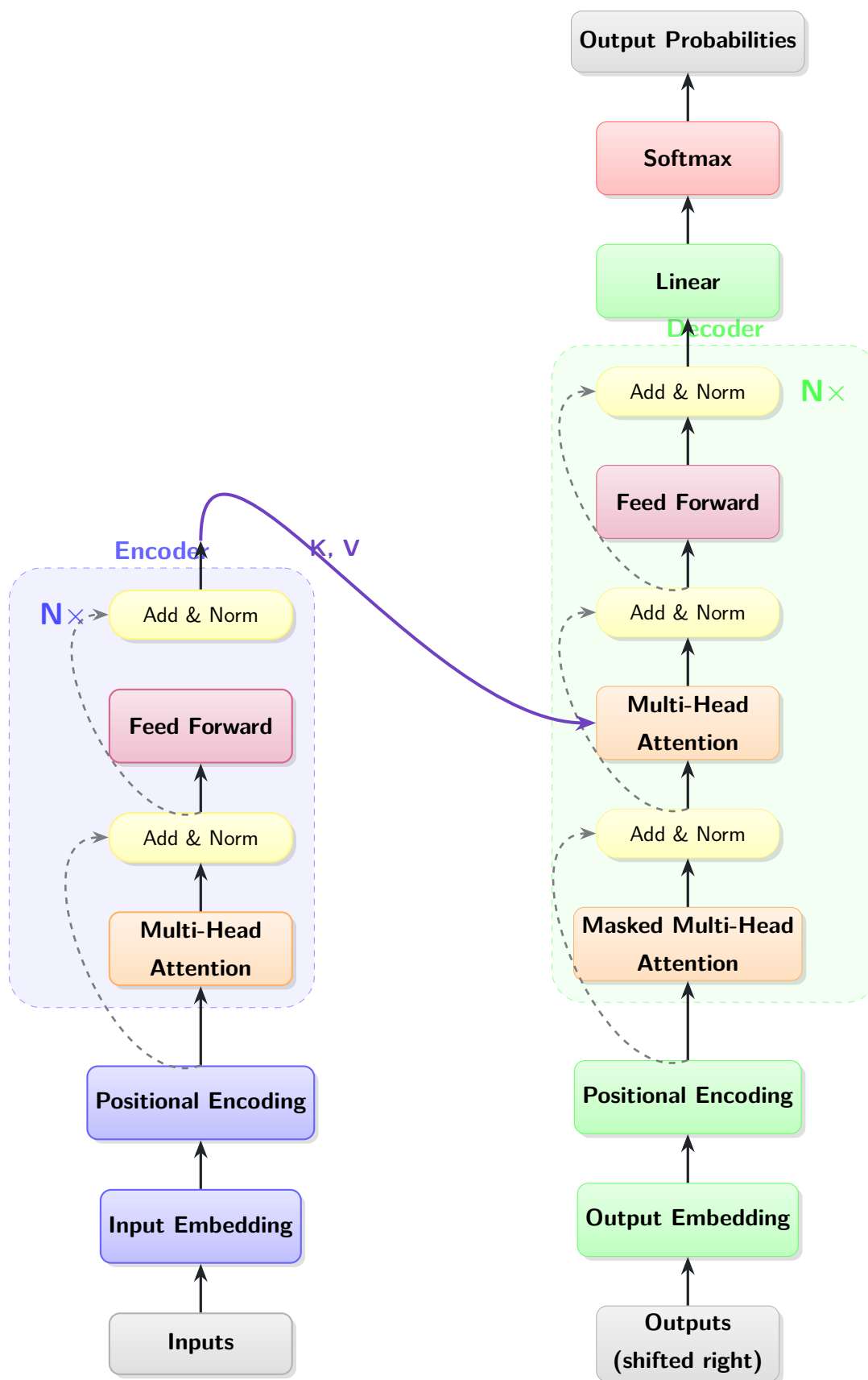


图 5: Transformer 模型整体架构图。左侧为编码器，右侧为解码器。